40. DOCKER

Images • Containers • Dockerfile • Volumes • Networks • Compose • ML Serving

Docker'ın temel mental modeli

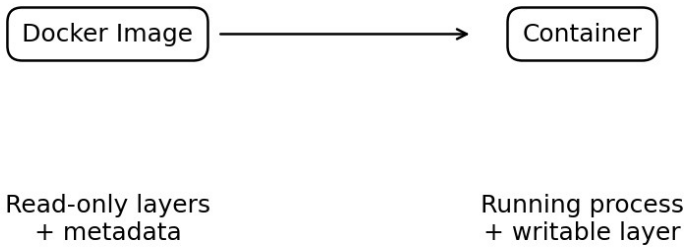


Image = şablon, Container = çalışan instance

Bu modülün amacı
Python, FastAPI ve ML projelerini dependency'leriyle birlikte taşınabilir bir container image haline getirmek; local geliştirme ile production ortamı arasındaki farkı azaltmak. Image/container mental modeli, Dockerfile yazımı, volumes, networking, Compose, güvenlik ve production optimizasyonları veri bilimci odaklı ele alınacak.

Kazanım	Modül sonunda
Mental model	Image, container, registry, layer ve runtime farklarını açıklayabilirsin.
Dockerfile	Python/ML uygulaması için tekrar üretilebilir image oluşturabilirsin.
Storage	Bind mount, named volume ve ephemeral filesystem farkını anlayabilirsin.
Networking	Container portları ve service-to-service communication kurabilirsin.
Compose	API + DB + cache gibi çok servisli ortamı tek config ile ayağa kaldırabilirsin.
Production	Image size, security, healthcheck, non-root user ve secrets konularını yönetebilirsin.

1. Docker neden var?

Bir uygulama yalnız Python dosyalarından oluşmaz. Python sürümü, sistem kütüphaneleri, package versiyonları, environment variables ve çalıştırma komutu da davranışı belirler. Docker bu runtime ortamını image olarak paketler.

Problem	Docker yaklaşımı
Bende çalışıyor, sende çalışmıyor	Aynı image
Dependency çakışması	İzole filesystem/process
Deploy ortamı farklı	Portable runtime
Kurulum adımları manuel	Dockerfile ile deklaratif build

Ana fikir
Docker uygulamayı sanal makine gibi tam OS ile değil, izole process + filesystem + network olarak paketler.

2. Image vs Container

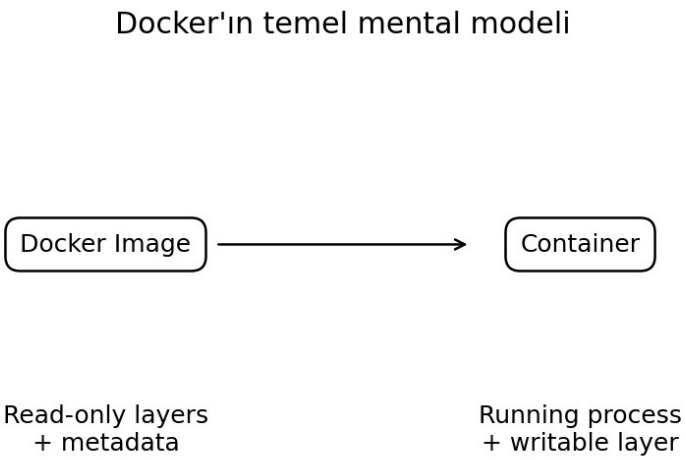


Image	Container
Immutable şablon	Image'ın çalışan instance'ı
Build edilir	Run edilir
Registry'de saklanır	Host üzerinde process olarak çalışır
Layer'lardan oluşur	Ek writable layer alır

3. Docker Engine

- Docker CLI
- Docker daemon / engine
- Images

- Containers
- Networks
- Volumes

CLI ne yapar?
docker build/run/push gibi komutlar engine ile konuşur.

4. Container VM değildir

VM	Container
Guest OS taşır	Host kernel paylaşır
Daha ağır	Daha hafif
Boot daha yavaş	Startup hızlı
Güçlü isolation	Process/container isolation

Pratik sonuç
Aynı host üzerinde daha fazla container çalıştırılabilir.

5. İlk container

```
docker run --rm hello-world
```

Flag	Anlam
run	Image'dan container başlat
--rm	Container durunca otomatik sil
hello-world	Image adı

6. Image pull

```
docker pull python:3.12-slim
```

Tag
python:3.12-slim içindeki 3.12-slim tag'dir. latest kullanımı production reproducibility için riskli olabilir.

7. Container listesi

```
docker ps
docker ps -a
```

Stopped container
Container durmuş olsa da metadata/filesystem state silinmemiş olabilir.

8. Container lifecycle

```
docker start my_container
docker stop my_container
docker restart my_container
docker rm my_container
```

9. Dockerfile nedir?

Image'ın nasıl build edileceğini tanımlayan deklaratif dosyadır.

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "main.py"]
```

Dockerfile = build recipe
Container çalıştığında hangi filesystem ve default command olacağını belirler.

10. FROM

```
FROM python:3.12-slim
```

Base image
Genellikle language runtime veya minimal Linux base üstüne build edilir.

11. WORKDIR

```
WORKDIR /app
```

Fayda
Sonraki COPY/RUN/CMD işlemleri için çalışma dizinini belirler.

12. COPY ve ADD

Komut	Kullanım
COPY	Dosyaları image içine kopyala
ADD	Ek özellikler içerir; çoğu durumda COPY daha açık

Genel tercih
Dosya taşımada COPY kullan; ADD'e gerçekten ihtiyacın varsa seç.

13. RUN

```
RUN pip install    --no-cache-dir    -r requirements.txt
```

Build-time
RUN image build sırasında çalışır ve sonuç layer'a yazılır.

14. CMD vs ENTRYPOINT

CMD	ENTRYPOINT
Default command/args	Container executable'ını sabitler
Override kolay	Args ile birlikte kullanılabilir

```
CMD [
  "uvicorn",
  "app.main:app",
  "--host", "0.0.0.0",
  "--port", "8000"
]
```

15. EXPOSE

```
EXPOSE 8000
```

Önemli
EXPOSE portu host'a yayınlamaz; image metadata/documentation gibidir. Host mapping için -p gerekir.

16. ENV

```
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
```

Secret değil
Dockerfile içine password/API key yazma. ENV image history içinde kalabilir.

17. Build

```
docker build   -t churn-api:1.0 .
```

Parça	Anlam
-t	Image adı/tag
churn-api:1.0	Repository:tag
.	Build context

18. Build context

docker build sonunda verdiğin directory, Docker daemon/build engine'e gönderilen context'tir.

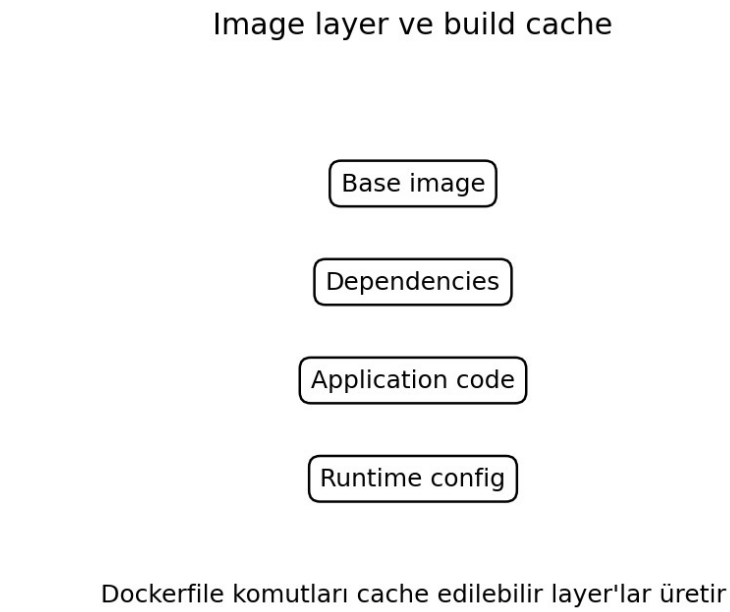
Büyük context
data/, .git/, notebooks/, model cache gibi gereksiz dosyalar build'i yavaşlatabilir.

19. .dockerignore

```
.git
.venv
__pycache__
*.pyc
data/
notebooks/
.env
```

Fayda
Build context küçülür; secret ve gereksiz dosyaların image'a yanlışlıkla girmesi azalır.

20. Image layers



Dockerfile'daki bazı instruction'lar layer üretir; değişmeyen layer cache'den yeniden kullanılabilir.

21. Cache-friendly Dockerfile

```
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY src/ ./src/
```

Neden?
Application code her değiştiğinde dependency installation layer'ını invalid etmek istemezsin.

22. Image history

```
docker history churn-api:1.0
```

Kullanım
Hangi layer'ın image size'a ne kadar katkı yaptığını gör.

23. Slim base image

Image	Trade-off
python:3.12	Daha büyük, daha fazla system package
python:3.12-slim	Küçük ve production için yaygın
Alpine	Çok küçük ama native dependency uyumluluğu zorlaşabilir

ML dünyası
NumPy/PyTorch gibi native paketlerde aşırı minimal base image build karmaşasını artırabilir.

24. Multi-stage build

```
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip wheel    --wheel-dir /wheels    -r requirements.txt

FROM python:3.12-slim
COPY --from=builder /wheels /wheels
RUN pip install /wheels/*
COPY . /app
WORKDIR /app
```

Amaç
Compiler/build tool'larını final runtime image'a taşımamayı sağlar.

25. Container çalıştırma

```
docker run  --rm  -p 8000:8000  churn-api:1.0
```

Port mapping
host:container mantığıdır. 8000:8000 -> host 8000, container 8000.

26. Detached mode

```
docker run -d  --name churn-api  -p 8000:8000  churn-api:1.0
```

27. Logs

```
docker logs churn-api
docker logs -f churn-api
```

Container logging
Uygulama loglarını stdout/stderr'e yazmak container platformlarıyla uyumludur.

28. Exec

```
docker exec -it churn-api sh
```

Debug
Çalışan container'ın içine shell ile girip filesystem/process durumunu inceleyebilirsin.

29. Ephemeral filesystem

Container silindiğinde writable layer içindeki data da silinir.

Kural
Kalıcı database/data dosyalarını container writable layer'ına güvenerek saklama.

30. Volumes

Tür	Kullanım
Named volume	Docker-managed persistent storage
Bind mount	Host path'i container'a mount et
tmpfs	Memory-backed geçici storage

31. Named volume

```
docker volume create pgdata

docker run -v pgdata:/var/lib/postgresql/data postgres
```

Database
Local development database persistence için yaygın.

32. Bind mount

```
docker run -v "$(pwd)/src:/app/src" churn-api:dev
```

Development
Kod değişikliğini image rebuild etmeden container içinde görmek için kullanılabilir.

33. Environment variables

```
docker run -e MODEL_PATH=/models/model.joblib -e LOG_LEVEL=INFO churn-api:1.0
```

Config injection
Aynı image dev/test/prod için farklı environment ile çalışabilir.

34. env-file

```
docker run   --env-file .env   churn-api:1.0
```

Güvenlik
env-file repo'ya commit edilmemeli; production'da secret manager/orchestrator secret mekanizması tercih edilir.

35. Secrets

- Dockerfile'a secret yazma.
- Build arg ile secret bake etme.
- Git repository'ye .env koyma.
- Runtime secret injection kullan.

Image immutable olsa da
Image registry veya build cache'de secret kalıcı iz bırakabilir.

36. Docker networking

Docker networking sezgisi



Container'lar user-defined network içinde servis adıyla konuşabilir

Default bridge vs user-defined network
User-defined network DNS/service-name resolution ve daha kontrollü isolation sağlar.

37. User-defined network

```
docker network create app-net

docker run -d   --network app-net   --name db   postgres

docker run -d   --network app-net   --name api   churn-api:1.0
```

DNS
api container DB'ye hostname olarak 'db' ile erişebilir.

38. localhost tuzağı

Container içindeki localhost o container'ın kendisidir; host veya başka container değildir.

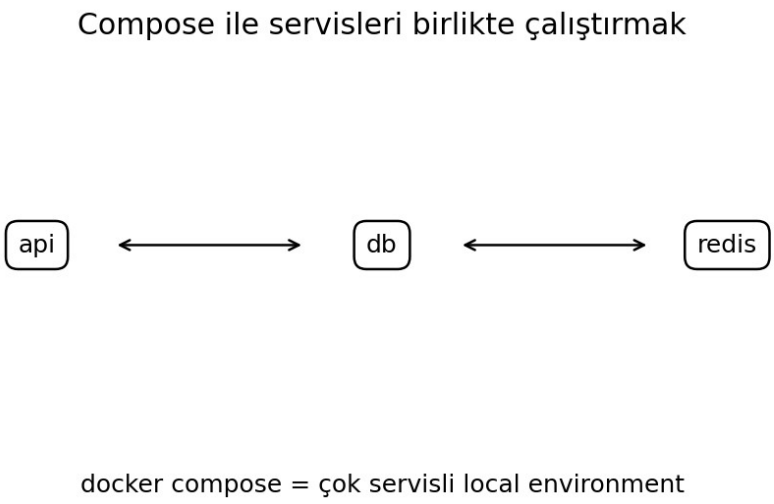
Çözüm
Başka container için service name/network; host için platforma uygun host gateway kullan.

39. DNS ve service names

Compose veya user-defined network altında container'lar servis/container adıyla birbirini bulabilir.

```
DB_HOST=db
REDIS_HOST=redis
```

40. Docker Compose



Birden çok container, volume ve network'ü tek YAML dosyasında tanımlamayı sağlar.

41. Compose örneği

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      DB_URL: postgresql://db/app
    depends_on:
      - db

  db:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Local stack
API + DB + Redis + worker gibi dependency'leri tek komutla ayağa kaldırmak için güçlü.

42. Compose komutları

```
docker compose up -d
docker compose ps
docker compose logs -f
docker compose down
```

43. depends_on sınırı

depends_on container start sırasını ifade eder; downstream servis gerçekten hazır mı sorusunu çözmeyebilir.

Readiness
DB portu açılmış olsa bile migration tamamlanmamış olabilir. Healthcheck/retry logic gerekir.

44. Healthcheck

```
HEALTHCHECK --interval=30s --timeout=3s   CMD python -c   "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"
```

Platform entegrasyonu
Container orchestrator unhealthy container'ı yeniden başlatabilir veya traffic'ten çıkarabilir.

45. FastAPI container

```
FROM python:3.12-slim

WORKDIR /app

COPY pyproject.toml .
COPY src/ ./src/

RUN pip install .

CMD [
  "uvicorn",
  "src.app:app",
  "--host", "0.0.0.0",
  "--port", "8000"
]
```

0.0.0.0
Container dışından port'a erişebilmek için server sadece 127.0.0.1'e bind edilmemeli.

46. ML artifact image içine mi?

Yaklaşım	Artı	Eksi
Model image içinde	Atomic deploy	Image büyük, model update için rebuild
Model volume/object storage	Image küçük	Startup download/version yönetimi
Model server ayrı	Bağımsız scale	Mimari karmaşıklık

Seçim
Model boyutu, update sıklığı ve deployment topology'ye göre.

47. Model startup loading

Model container startup/lifespan sırasında bir kez yüklenmeli; her request'te diskten okunmamalıdır.

Ready değilse
Readiness false dönmeli; traffic ancak model load tamamlandıktan sonra başlamalı.

48. GPU container sezgisi

Container GPU driver'ı kendi içine tam olarak paketlemez; host runtime ile GPU device erişimi sağlanır.

ML notu
CUDA runtime, driver uyumluluğu ve PyTorch/TensorFlow image versiyonları birlikte düşünülmelidir.

49. Docker registry

Image'ların versioned olarak push/pull edildiği merkezi registry.

- Docker Hub
- Cloud registries
- Private registries

Production
Deployment host image'ı Git repository'den değil registry'den çeker.

50. Tagging

```
docker tag churn-api:1.0 registry.example.com/ml/churn-api:1.0
```

latest riski
latest hangi build olduğunu söylemez. Commit SHA veya semantic version daha izlenebilir.

51. Immutable tags

Aynı tag'e farklı image push edilmesini engelleyen registry policy.

Reproducibility
v1.2.3 veya git SHA aynı binary/image'ı işaret etmeli.

52. Push

```
docker push registry.example.com/ml/churn-api:1.0
```

53. Image digest

Image content'inin cryptographic identifier'ıdır.

En kesin referans
Tag değişebilir; digest belirli image içeriğini kesin işaret eder.

54. Non-root user

```
RUN useradd --create-home appuser

USER appuser
```

Güvenlik
Container process'i root olarak çalışmak gereksiz privilege riskidir.

55. Read-only filesystem

Uygulama runtime'da filesystem'e yazmak zorunda değilse read-only root filesystem security'yi artırabilir.

Write ihtiyacı
tmp veya upload için ayrı writable mount tanımla.

56. Linux capabilities

Root yetkisinin alt parçalara ayrılmış privilege mekanizması.

Least privilege
Gereksiz capabilities drop etmek container attack surface'i azaltır.

57. Resource limits

```
docker run   --memory=2g   --cpus=2   churn-api:1.0
```

Neden?
Tek container'ın host kaynaklarını tamamen tüketmesini önler; production scheduler'larda requests/limits benzeri kavramlar vardır.

58. OOM davranışı

Container memory limit'i aşarsa process OOM killed olabilir.

- Model çok büyük
- Worker sayısı fazla
- Batch fazla
- Memory leak

Debug
Container restart loop görürsen memory metrics ve exit code kontrol et.

59. PID 1 ve signal handling

Container içindeki ana process PID 1 olur; SIGTERM/SIGINT sinyallerini doğru işlemesi graceful shutdown için kritiktir.

JSON form CMD
CMD ["python", "app.py"] shell form'a göre signal forwarding için daha güvenli bir pattern'dir.

60. Graceful shutdown

- Yeni request kabul etmeyi bırak.
- Mevcut request'leri tamamla.
- Connections kapat.
- Logs flush et.

Orchestrator
Deploy sırasında eski container SIGTERM alıp belirli grace period içinde kapanır.

61. Image vulnerability scanning

Base image ve dependency'lerde bilinen CVE'leri taramak production security'nin parçasıdır.

Rebuild
Base image güncellendiğinde uygulama kodu değişmese bile security patch için image yeniden build edilmelidir.

62. Base image pinleme

```
FROM python:3.12.7-slim
```

Trade-off
Tam versiyon pinlemek reproducibility sağlar; security patch almak için düzenli rebuild/update gerekir.

63. Dependency pinleme

Python dependency versiyonları lock/pin edilmezse farklı tarihlerde farklı image üretilebilir.

Build reproducibility
requirements lock veya pyproject lock mekanizması kullan.

64. Cache ve package manager

```
RUN pip install  --no-cache-dir  -r requirements.txt
```

Image size
pip cache final image için gereksizse silmek size'ı azaltır.

65. Apt cleanup

```
RUN apt-get update  && apt-get install -y --no-install-recommends       libgomp1  && rm -rf /var/lib/apt/lists/*
```

Tek RUN
Install ve cleanup aynı layer'da yapılırsa eski package index layer'da kalmaz.

66. Reproducible builds

- Pinned base image
- Pinned dependencies
- Controlled build args
- Clean build context
- Deterministic artifact

Hedef
Aynı source revision mümkün olduğunca aynı image'ı üretmeli.

67. CI/CD ile Docker

```
on:
  push:
    branches: [main]

steps:
  - checkout
  - run tests
  - docker build
  - scan image
  - docker push
  - deploy
```

37 + 38 + 40
Git commit -> tests -> image -> registry -> deployment zinciri profesyonel delivery pipeline'ıdır.

68. BuildKit

Modern Docker build engine; paralel build, gelişmiş cache ve secret mounts gibi özellikler sağlar.

Secret mount
Build-time private package credential gerekiyorsa image layer'a bake etmeden kullanılabilir.

69. Development vs production image

Development	Production

Hot reload	Immutable code
Bind mounts	Copied package
Debug tools	Minimal runtime
Large convenience deps	Only required deps

Aynı olmak zorunda değil
Ama dependency/runtime farkı bug yaratmayacak kadar yakın tutulmalı.

70. Docker ve notebooks

Jupyter ortamını image ile standardize etmek reproducible development sağlayabilir.

```
docker run   -p 8888:8888   -v "$(pwd):/workspace"   ds-notebook:1.0
```

Data mount
Büyük dataset'i image içine COPY etmek yerine volume/object storage kullan.

71. Data science project Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY pyproject.toml .
COPY src/ ./src/
COPY artifacts/model.joblib ./artifacts/

RUN pip install .

ENV MODEL_PATH=/app/artifacts/model.joblib

CMD [
  "uvicorn",
  "src.api:app",
  "--host", "0.0.0.0",
  "--port", "8000"
]
```

72. docker compose ML stack

```
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      REDIS_URL: redis://redis:6379

  redis:
    image: redis:7-alpine
```

Local integration
Feature cache veya queue dependency'siyle birlikte geliştirme ve test kolaylaşır.

73. Batch job container

```
docker run --rm   -e DATE=2026-10-10   batch-scorer:1.0
```

Container sadece web API değildir
Scheduled ETL, batch inference, training job ve CLI task de container olarak paketlenebilir.

74. Training container

Training script + dependencies + runtime image'a paketlenir; data object storage'dan okunur, model artifact dış storage'a yazılır.

Cloud bağlantısı
Managed training platformları çoğu zaman kullanıcıdan bir container/image veya training package bekler.

75. MLflow / experiment container bağlantısı

- Training image version
- Git commit
- Dataset version
- Hyperparameters
- Artifact URI

MLOps
Image digest experiment metadata'ya yazılırsa runtime tam olarak izlenebilir.

76. Docker anti-patterns

- latest tag'e güvenmek.
- Secret'ı Dockerfile içine yazmak.
- Modeli her request'te yüklemek.
- Büyük dataset'i image içine COPY etmek.
- Root user ile çalışmak.
- Tek container içine API + DB + scheduler + her şeyi koymak.
- Healthcheck tanımlamamak.
- .dockerignore kullanmamak.
- Dependency'leri pinlememek.
- Image size'ı ölçmeden build etmek.

77. Debugging checklist

1. Container çalışıyor mu? docker ps
2. Exit code ne?
3. Logs ne söylüyor?
4. Port doğru map edildi mi?
5. Server 0.0.0.0'a bind mi?
6. Environment variable geldi mi?
7. Volume path doğru mu?
8. Network/service name doğru mu?
9. File permissions uygun mu?
10. Memory/CPU limit yetiyor mu?

78. Production checklist

1. Pinned base image.
2. Pinned dependencies.
3. Non-root user.
4. .dockerignore.
5. Secrets runtime injection.
6. Health/readiness checks.
7. Resource limits.
8. Structured stdout logs.
9. Image vulnerability scan.
10. Versioned immutable tag/digest.
11. Graceful shutdown.
12. Minimal final image.

79. Gerçek vaka: FastAPI churn service

Katman	Docker kararı
Base	python:3.12-slim
Dependencies	pyproject/lock first
Model	Image içine versioned artifact
Server	Uvicorn
Port	8000
Config	ENV/runtime
Health	/health/ready
Security	non-root user

80. Gerçek vaka: RAG local stack

Service	Container

API	FastAPI RAG service
Vector DB	Chroma/Qdrant benzeri
Cache	Redis
LLM	Remote API veya local model server
Observability	Logs/metrics collector

Compose
Bu servisleri local integration test için tek compose dosyasında birleştirebilirsin.

81. Gerçek vaka: scheduled batch scorer

Her gece object storage'dan feature parquet okuyup scoring yapar, sonuçları warehouse'a yazar.

Container avantajı
Scheduler'ın yalnız image + command + environment bilmesi yeterlidir.

82. Mülakat soruları

Soru	Kısa cevap
Image vs container?	Image immutable şablon; container çalışan instance.
Dockerfile ne?	Image build recipe.
RUN vs CMD?	Build-time command vs container default runtime command.
COPY neden önce dependency dosyası?	Layer cache reuse.
EXPOSE port publish eder mi?	Hayır; host mapping -p ile.
Volume neden?	Container lifecycle'dan bağımsız kalıcı data.
Bind mount ne?	Host path'i container'a mount etmek.
localhost tuzağı?	Container içindeki localhost o container'ın kendisi.
Compose ne için?	Çok servisli local stack tanımı.
Non-root user neden?	Least privilege/security.
Multi-stage build?	Build dependency'lerini final image'dan ayırmak.
Healthcheck?	Container/application health sinyali.
latest neden riskli?	Hangi image olduğunu kesin belirtmez.
Image digest?	Content-addressed immutable identifier.
Docker VM mi?	Hayır; host kernel paylaşan izole process modeli.

83. Uygulama alıştırmaları

1. Basit Python script'i Dockerfile ile containerize et.
2. python:3.12 ve python:3.12-slim image size'larını karşılaştır.
3. .dockerignore ekle ve build context farkını gözlemle.
4. Dependency cache için COPY sırasını optimize et.
5. FastAPI uygulamasını -p 8000:8000 ile çalıştır.
6. Bind mount ile hot-reload development ortamı kur.
7. Named volume ile Postgres data persistence dene.
8. API + Postgres için docker compose yaz.
9. Container'ı non-root user ile çalıştır.

10. Image vulnerability scan + size raporu oluştur.

84. Mini portföy projesi

Proje: Containerized Production ML Service

Dosya	İçerik
Dockerfile	Multi-stage/minimal production build
.dockerignore	Clean build context
compose.yaml	API + Redis/Postgres
src/api.py	FastAPI serving
src/service.py	Model inference logic
artifacts/	Versioned model
tests/	Container smoke + API tests
.github/workflows/ci.yml	Test + build + scan
README.md	Build/run/architecture/security

CV cümlesi
Containerized a production ML API with Docker using cache-efficient multi-stage builds, non-root execution, runtime configuration, health checks, persistent volumes, service networking, Docker Compose integration and CI-based image scanning.

85. Final cheat sheet

Kavram	Hatırlanacak fikir
Image	Immutable runtime template
Container	Running image instance
Dockerfile	Build recipe
Layer	Cached filesystem change
Build context	docker build input set
.dockerignore	Context/filter/security
RUN	Build time
CMD	Runtime default
Port mapping	host:container
Volume	Persistent external storage
Bind mount	Host path mapping
Network	Container communication
Compose	Multi-service definition
Registry	Image storage/distribution
Tag	Human version label
Digest	Immutable content identity
Non-root	Least privilege
Healthcheck	Runtime health signal
Multi-stage	Small final image
Resource limit	CPU/memory guard

Bu modülden sonra
41. MLOps bölümünde Docker image'larını experiment tracking, model registry, versioning, pipeline ve deployment yaşam döngüsüyle birleştireceksin.